
Kotlin

VARIABLES

Una variable no es más que un espacio de memoria en el que podemos guardar información. Dependiendo del tipo de información disponemos de diferentes variables, lo que nos permitirá evitar problemas como por ejemplo intentar sumar un número con una letra, ya que al ser tipos de variables diferentes no nos lo permitirá.

Las variables las declararemos del siguiente modo:

var numeroFavorito = 1

«var» nos dice que es una variable, a continuación, le asignamos un nombre, en este caso

«numeroFavorito» y le asignamos un valor.

Aunque no le hemos dicho el tipo de variable que es, Kotlin lo acepta, esto es porque Kotlin busca el tipo de valor y le asigna un tipo por detrás, el problema de esto es que no siempre estará en lo correcto, así que debemos decirle nosotros el tipo.

var numeroFavorito: Int = 1

Esta vez le hemos puesto dos puntos y acto seguido hemos añadido el tipo de variable, en este caso Int. Ahora veremos con que variables podemos trabajar.

VARIABLES

Kotlin maneja dos tipos de variables:

- **mutables**: son aquellas que pueden cambiar de valor. Se definen usando el modificador **var**.
- **inmutables**: son aquellas que no pueden cambiar de valor (es similar a las variables final en Java). Se definen usando el modificador **val**.

VARIABLES numéricas

Se dividen en dos grupos, las variables **enteras** y **reales**.

- **Integer**. Con una variable de tipo Int podemos declarar valores entrede (-2,147,483,647 , 2,147,483,647).

  var numeroFavorito: Int = -231

- **Long**. Básicamente es igual que Int, a diferencia de el dominio es un rango mayor de números (-9,223,372,036,854,775,807, 9,223,372,036,854,775,807).

  var numeroFavorito: Long = 47483647

- **Float**. Pueden almacenar decimales. Float soporta hasta 6 decimales, pero también puede trabajar con números enteros. Esta variable cambia un poco respecto a las demás, pues habrá que meter una «f» al final del valor.

  var numeroFavorito: Float = 1.93f

- **Double**. Muy similar a float pero soporta hasta 14 decimales, pero también ocupa más memoria así que para un código óptimo deberemos pensar que tipo será el que más se adapte a nuestro proyecto. Tampoco habrá que añadir ningún tipo de letra al final del valor.

  var numeroFavorito: Double= 1.932214124

VARIABLES alfanuméricas y booleanas

- **Char**. Nos permite guardar un carácter de cualquier tipo, lo único que debemos tener en cuenta es que va entre comillas simples.

  var letraFavorita: Char = ‘q’

- **String.** La variable String será la que más usemos como norma general, nos permite almacenar cualquier tipo de caracteres, pero a diferencia del Char, podemos añadir la cantidad que queramos. Para ser exactos, una String no es más que una cadena de Char. Las cadenas deberán ir entre comillas dobles.

  var numeroFavorito: String = "Mi número favorito es el 3“

- **Boolean.** Los Booleanos son variables que solo pueden ser verdaderas o falsas (true o false). Su uso es muy amplio, cuando trabajemos con las condiciones veremos más a fondo este tema. Para asignar un valor basta con añadir true o false sin comillas.

  var estoyTriste: Boolean = false

La clase Any

Es la clase raíz de toda la jerarquía de clases.

Esto significa que cada clase en Kotlin, excepto la propia Any, es una subclase de Any de forma implícita.

Una variable declarada como tipo Any puede contener un valor de cualquier otro tipo, ya sea un String, un Int, un Boolean o una instancia de cualquier otra clase que hayamos definido.

```
fun main() {
     val a: Any = 10
     val b: Any = 10
     val c: Any = "10"

     // Compara si el objeto 'a' es igual al objeto 'b'
     val sonIguales = a.equals(b)
     println("¿a y b son iguales? $sonIguales")

     // Compara si el objeto 'a' es igual al objeto 'c'
     val sonIguales2 = a.equals(c)
     println("¿a y c son iguales? $sonIguales2")
}
// Salida:
// ¿a y b son iguales? True
// ¿a y c son iguales? false
```

Any proporciona una base para todos los objetos, y sus métodos básicos, como equals(), hashCode() y toString(), están disponibles para cualquier objeto en Kotlin.

OPERADOR $

El operador del signo de dólar ($) en Kotlin se utiliza para la interpolación de cadenas (string templates). Permite insertar variables, expresiones o el resultado de una función directamente dentro de una cadena de texto. Ejemplos de uso:

Para insertar una variable	val nombre = "Juan" println("Hola, $nombre") // Salida: Hola, Juan
Para insertar el resultado de una expresión (hemos de ponerla entre {})	val a = 10 val b = 5 println("La suma de $a y $b es ${a + b}") // Salida: La suma de 10 y 5 es 15
Para insertar un método o una propiedad	data class Persona(val nombre: String)val persona = Persona("María") println("El nombre tiene ${persona.nombre.length} letras") // Salida: El nombre tiene 5 letras

OPERADOR ?

El operador **?** en Kotlin se usa para la seguridad contra valores nulos (null safety). Es una característica fundamental del lenguaje que ayuda a prevenir errores comunes y fallos en las aplicaciones, como el NullPointerException de Java.

Se coloca después del tipo de una variable para declarar que esa variable puede contener un valor nulo. Esto le indica al compilador que debe tratar esa variable con precaución.

Permite llamar a un método o acceder a una propiedad de un objeto solo si el objeto no es null. Si la variable es null, la llamada simplemente no se ejecuta y la expresión completa devuelve null, en lugar de fallar.

OPERADOR ?

```
fun main(){

    var nombre: String? = "Alice"

    var longitud = nombre?.length
        println("La longitud de $nombre es $longitud")

    nombre = null

    // Usa el operador ?. para llamar a la función sin que falle
    longitud = nombre?.length
    println("La longitud de $nombre es $longitud")
}
    // Salida:
    // La longitud de Alice es 5
    // La longitud de null es null
```

OPERADOR !!

El operador **!!** en Kotlin (el doble signo de exclamación) se llama operador de aserción no nula (Non-null Assertion Operator).

Cuando tenemos variables que pueden ser nulas, para evitar errores que lancen excepciones Kotlin tiene que asegurarse de que cuando se van a usar esas variables, por ejemplo, en expresiones aritméticas, estas no son null en ese momento.

Es una forma de decirle a Kotlin que estoy seguro de que en este momento de la ejecución estas variables no van a ser null porque las he validado previamente.

Kotlin va a confiar, pero no lo verifica. Es decir, si en algún momento alguna de las variables toma el valor null, el programa fallará inmediatamente con un error grave en tiempo de ejecución (NullPointerException en Java Virtual Machine).

OPERADOR !!

```
fun main(){

    var valor: String? = null

    val longitud = valor.length
    println(longitud)
}
```

No deja ejecutarlo

```
fun main(){

    var valor: String? = null

    val longitud = valor!!.length
    println(longitud)
}
```

Da error de ejecución

```
fun main(){
    var valor: String? = null
    valor = "Ya no es null."
    val longitud = valor!!.length
    println(longitud)
}
```

OK

```
fun main(){

    var num1: Double?
    var doble: Double

    println("Por favor, introduce un número: ")
    num1= readln()?.toDoubleOrNull()
    doble = num1 * 2
    println("el doble de $num1 es: $doble")
}
```

```
fun main(){

    var num1: Double?
    var doble: Double = 0.0

    println("Por favor, introduce un número: ")
    num1= readln()?.toDoubleOrNull()
    if (num1 != null) doble = num1 * 2
    println("el doble de $num1 es: $doble")
}
```

Smart Cast

OPERACIONES ARITMETICAS

Suma, resta multiplicación y resto de la división entera:

```
fun main(args: Array<String>) {
    var a = 10
    var b = 5

    print("Suma: ")
    println(a + b)
    print("Resta: ")
    println(a - b)
    print("Multiplicación: ")
    println(a * b)
    print("División: ")
    println(a / b)
    print("El módulo (resto): ")
    println(a % b)
}
```

OPERACIONES RELACIONALES Y LÓGICOS

- Operadores relacionales:

```
> (mayor)
< (menor)
>= (mayor o igual)
<= (menor o igual)
== (igual)
!= (distinto)
```

- Operadores lógicos:

```
&& (and)
|| (or)
 ! (not).
```

OPERACIONES ARITMETICAS

Operaciones con variables de diferente tipo:

```
fun main(args: Array<String>) {
    var a: Float = 11.5f
    var b: Int = 3

    print("Suma: ")
    var resultado = a + b
    print(resultado)
}
```



En Kotlin, cuando se realiza una operación entre tipos numéricos diferentes, el compilador promociona automáticamente el tipo de dato más pequeño al más grande.
En este ejemplo ***b*** (Int) se convierte implícitamente en un Float para poder realizar la operación con la variable ***a***.
La operación se convierte en 11.5f + 3.0f, dando como resultado 14.5f, que se almacena en la variable ***resultado***, que es de tipo Float.

Esto no funciona. Si la variable ***resultado*** se ha definido como Int, no puedo sumar un float con un Int y asignarle la suma a esa variable, pues ya hemos visto que sería un Float

Esto si funciona. En Kotlin tenemos varios métodos **toX()**, para adaptar los tipos de las variables. En este caso se hace uso del método *toInt()*.

```
fun main(args: Array<String>) {
    var a: Float = 11.5f
    var b: Int = 3
    var resultado: Int

    print("Suma: ")
    resultado = a + b

    print("Suma: ")
    resultado = a.toInt() + b

    print(resultado)
}
```

OPERADOR toX()

Los tipos numéricos como Byte, Short, Int, Long, Float y Double tienen un conjunto de métodos toX() que te permiten convertirlos entre sí. Esto es útil para evitar errores de tipo en las operaciones.

toByte(): Convierte a Byte.

toShort(): Convierte a Short.

toInt(): Convierte a Int.

toLong(): Convierte a Long.

toFloat(): Convierte a Float.

toDouble(): Convierte a Double.

toChar(): Convierte a Char.

Las conversiones de tipos (toX()) solo están disponibles entre tipos numéricos y de caracteres que tienen una relación jerárquica clara.

Por ejemplo, podemos convertir un Int a un Double (10.toDouble()) o un Char a un Int ('A'.toInt()).

Sin embargo, no podemos convertir directamente un Int a un String usando un método toInt() ni convertir un String a un Int de esta manera. Para esas conversiones, se necesitan métodos específicos o diferentes enfoques.

OPERADOR toX()

Ejemplos:

```
fun main() { // EJEMPLOS
    val numeroInt = 10
    val numeroLong = numeroInt.toLong() // Convierte Int a Long

    val numeroDouble = 5.75
    val numeroFloat = numeroDouble.toFloat() // Convierte Double a Float

    val letra = 'A'
    val codigoAscii = letra.toInt() // Convierte Char a Int (su valor ASCII)

    println("El número Long es: $numeroLong")
    println("El número Float es: $numeroFloat")
    println("El código ASCII de 'A' es: $codigoAscii")
}
// Salida:
// El número Long es: 10
// El número Float es: 5.75
// El código ASCII de 'A' es: 65
```

OTRAS CONVERSIONES DE TIPOS

De Numérico a cadena/char

- **toString()**: convierte un valor numérico a String.
- **toChar()**: interpreta el número como el valor Unicode (ASCII) del carácter.

```
val numerostr = 123
val cadena1 = "El número es: $numerostr" // El Int se convierte a String automáticamente
val cadena2 = numerostr.toString()
val cadena3 = 456.toString()
println(cadena1)
println(cadena2)
println(cadena3)
val numeroch = 65
val caracter = numeroch.toChar()
println("El número $numeroch se convierte en el carácter: $caracter")
val otroCaracter = 97.toChar()
println("El número 97 se convierte en el carácter: $otroCaracter")
// Salida:
// El número es: 123
// 123
// 456
// El número 65 se convierte en el carácter: A
// El número 97 se convierte en el carácter: a
```

OTRAS CONVERSIONES DE TIPOS

cadena/char a numérico

- **cadena.toInt(), cadena.toDouble(), ...**: convierte un String al correspondiente tipo numérico. Para cada uno de estos métodos hay una versión 'safe' para controlar conversiones que podrían fallar. Estos métodos devuelven un valor nulo (null) en lugar de lanzar una excepción, lo que permite manejar errores de manera controlada. Estas funciones son:

  cadena.toIntOrNull(), cadena.toDoubleOrNull(), ...

- **caracter.toInt()**: Para convertir un carácter a un número, se usa el método toInt(). Este método devuelve el valor Unicode (ASCII) del carácter..

```
fun main() {
    val cadenaCorrecta = "123"
    val numero1 = cadenaCorrecta.toIntOrNull() // Devuelve 123
    val cadenaIncorrecta = "abc"
    val numero2 = cadenaIncorrecta.toIntOrNull() // Devuelve null

    val caracter = 'A'
    val valorNumerico = caracter.toInt() // Devuelve 65
}
```

CONCATENACIÓN

Para concatenar cadenas, debemos añadir las variables de forma consecutiva en el orden que queremos que se concatenen y englobar todas ellas entre comillas dobles. Para acceder al contendo de las variables usaremos el operador $ y si queremos que las cadenas aparezcan separadas por algún carácter como el espacio, debemos añadirlo.

```
fun main(args: Array<String>) {
    val greeting = "Hola, soy la "
    val name = “PMP Profe"
    println("$greeting $name")
}
//Salida: Hola, soy la PMP Profe
```

NOTA: En este ejemplo hemos usado variables inmutables. Siempre que preveamos que el valor no va a cambiar, intentaremos usar este tipo de variable.

COMPARACIÓN DE String

En Kotlin, la comparación de cadenas de texto (Strings) se realiza de dos maneras principales: comparando su contenido y comparando su referencia en memoria.

Comparación por Contenido (Igualdad Estructural)

Para comparar el contenido de do cadenas podemos usar el operador de igualdad (==) . También podemos compara el contenido obviando mayúsculas y minúsculas mediante el método **equals()** con el parámetro *ignoreCase* establecido en true.

```
val cadena1 = "hola“
val cadena2 = "hola“
val cadena3 = "Hola“
println(cadena1 == cadena2)
println(cadena1 == cadena3) //diferencia de mayúsculas y minúsculas
println(cadena1.equals(cadena3, ignoreCase = true))
// Salida:
// true
// false
// true
```

COMPARACIÓN DE String

Comparación por Referencia (Igualdad Referencial)

Para comparar si dos variables de tipo String apuntan al mismo objeto en la memoria, se utiliza el operador de igualdad referencial (===).

```
val a = "hola"
val b = "hola"
val c = a

println(a === b)
// Salida: true (Kotlin optimiza/reutiliza el mismo objeto para literales idénticos)
println(a === c)
// Salida: true (a y c apuntan a la misma referencia)

val cadenaNueva = String("hola".toCharArray())
println(a === cadenaNueva)
// Salida: false (Son objetos diferentes en la memoria, aunque el contenido sea idéntico)
```

DECLARACIÓN DE FUNCIONES

Las funciones se declaran usando la palabra clave fun, seguida del nombre del método, los paréntesis donde declararemos los valores de entrada y unas llaves que limitan la función. Al igual que en Java, las tres funciones Show… no se ejecutarán a no ser que se las invoque desde el **main**.

```
fun main(args: Array<String>) {
     showMyName()
     showMyLastName()
     showMyAge()
}

fun showMyName(){
     println("Mi nombre es Marta")
}
fun showMyLastName(){
     println("Mi Apellido es Rodríguez")
}
fun showMyAge(){
     println("Tengo 25 años")
}
// Salida:
// Mi nombre es Marta
// Mi Apellido es Rodríguez
// Tengo 25 años
```

DECLARACIÓN DE FUNCIONES

Funciones con parámetros de entrada :

```
fun main(args: Array<String>) {
     showMyInformation(“Marta ", "Rodríguez", 25)
}
fun showMyInformation(name: String, lastName: String, age: Int){
     println("Me llamo $name $lastName y tengo $age años.")
}
//Salida
// Me llamo Marta Rodríguez y tengo 25 años.
```

Funciones con parámetros de salida :

```
fun main(args: Array<String>) {
     var result = fadd(5, 10)
     println(result)
}
fun fadd(firsNumber: Int, secondNumber: Int) : Int{
     return firsNumber + secondNumber
}
//Salida
// 15
```

En el segundo caso, al tener sólo la instrucción return, se puede simplificar y eliminar las llaves y el return:

```
fun fadd(firsNumber: Int, secondNumber: Int) : Int = firsNumber + secondNumber
```

ENTRADA DE DATOS POR CONSOLA

En Kotlin, la entrada de datos por consola se realiza principalmente a través de la función **readln().** Esta función lee una línea completa de texto que el usuario introduce hasta que presiona la tecla "Enter".

La función **readln()** devuelve siempre un tipo String. Si neceitamos un tipo de dato diferente (como un Int o un Double), tendremos que convertir la cadena de texto de forma explícita.

Al igual que las funciones de conversión, esta función dispone de una versión safe que evita que el usuario pulse enter sin introducir información, es **readlnOrNull()**. Esta función, devolverá null si la entrada es vacía, lo cual permite manejar esta situación sin excepciones.

Como en ocasiones tendremos que convertir el String a otro tipo, haremos uso del operador '?' para prevenir un problema de conversión. Si readlnOrNull() devuelve null, la llamada a .toInt() se omite y la variable edad se mantiene como null.

```
fun main() {
    println("Introduce tu edad:")
    val edad = readlnOrNull()?.toInt()

    if (edad != null) {
        println("Tu edad es $edad.")
    } else {
        println("Entrada no válida.")
    }
}
```

CONDICIONAL if

- **IF como expresión** que devuelve un **valor**. Es equivalente al operador ternario de Java.

```
// Versión en Kotlin
val nota = 7
val resultado = if (nota >= 5) "Aprobado" else "Suspenso"
// resultado toma el valor "Aprobado”
```

```
// Versión en Java con el operador ternario
int nota = 7;
String resultado = (nota >= 5) ? "Aprobado" : "Suspenso";
```

- **IF como sentencia**, usando bloques {} y con opción de if-else.

```
// Versión en Kotlin
val edad = 18
if (edad >= 18) {
     println("Puedes votar.")
} else {
     println("No puedes votar.")
}
```

```
// Versión en Java
int edad = 18;
if (edad >= 18) {
     System.out.println("Puedes votar.");
} else {
     System.out.println("No puedes votar.");
}
```

ESTRUCTURA when

La estrutura **when** es similar al **switch** de Java. Aunque ambos se utilizan para seleccionar un bloque de código a ejecutar basado en el valor de una variable, el when permite una lógica más compleja y elimina la necesidad de los *breaks.*

Adicionalmente, la estructura when no solo es una sentencia, sino que también es una expresión, lo que significa que puede devolver un valor.

When puede comparar un valor de una variable con casos específicos:

```
Val numero = 2
val resultado = when (numero) {
                    1 -> "uno"
                    2 -> "dos"
                    else -> "otro“
}
println(resultado)
// Salida: dos
```

ESTRUCTURA when

When puede comparar un valor de una variable con rangos separados por comas:

```
fun getMonth(month : Int){
     when (month) {
          1,2,3 -> print("Primer trimestre del año")
          4,5,6 -> print("segundo trimestre del año")
          7,8,9 -> print("tercer trimestre del año")
          10,11,12 -> print("cuarto trimestre del año")
          else -> print(“mes no válido")
     }
}
```

ESTRUCTURA when

When puede comparar un valor de una variable con rangos (**in** a..b) y (**!in** a..b) :

```
fun mostrarMensaje(puntuacion: Int, tienePremio: Boolean) {
    when (puntuacion) {
        in 0..99 -> println("Sigue practicando.")
        in 100..499 -> {
            println("¡Casi lo consigues!")
        500 -> println("Has logrado llegar a 500.")
        in 501..1000 -> {
            if (tienePremio) {
                println("Has logrado el premio.")
            } else {
                println("¡Excelente puntuación! Sigue así y lograrás el premio.")
            }
        }
        else -> println("Puntuación fuera de rango.")
    }
}

fun main() {
    mostrarMensaje(250, false) // Salida: ¡Casi lo consigues!
    mostrarMensaje(500, false) // Salida: Has logrado llegar a 500.
    mostrarMensaje(800, true) // Salida: Has logrado el premio.
}
```

ESTRUCTURA when

Puede comparar un valor de una variable con tipos de datos (**is** Tipo).

```
fun result(value: Any){
     when (value){
          is Int -> print(value + 1)
          is String -> print("El texto es $value")
          is Boolean -> if (value) print("es verdadero") else print("es falso")
          else -> print("Tipo de dato no reconocido")
     }
}
```

ESTRUCTURA when

Otro ejemplo de comparar un valor de una variable con tipos de datos (**is** Tipo).

```
fun describir(obj: Any): String =
    when (obj) {
        1 -> "El objeto es un ${obj::class.simpleName} y su valor es 1"
        "Hola" -> "El objeto es una ${obj::class.simpleName} y su valor es 'Hola'"
        is Long -> "El objeto es un ${obj::class.simpleName} y es un número largo"
        !is String -> "El objeto no es una cadena de texto, sino un ${obj::class.simpleName}"
        else -> "El objeto es una ${obj::class.simpleName} con otro valor"
    }

fun main() {
    println(describir(1))
    println(describir("Hola"))
    println(describir(100L))
    println(describir(true))
    println(describir("Adiós"))
}
//Salida:
// El objeto es un Int y su valor es 1
// El objeto es una String y su valor es 'Hola’
// El objeto es un Long y es un número largo
// El objeto no es una cadena de texto, sino un Boolean
// El objeto es una String con otro valor
```

ESTRUCTURA when

Podemos guardar el resultado de un when automáticamente.

```
fun result(month : Int){
     val response : String = when (month) {
          in 1..6 -> "Primer semestre"
          in 7..12 -> "segundo semestre"
          !in 1..12 -> "no es un mes válido"
          else -> "error"
     }
}
```

Hemos declarado la variable de tipo String, pero podríamos hacerla de tipo Any si no tuviéramos claro el resultado de la expresión. Aquí si es obligatorio añadir un else.

Ejercicios propuestos.

Ejercicio 1: Crea un programa en Kotlin que solicite la entrada de cuatro notas por teclado y calcule su promedio.

Una vez obtenido el promedio, el programa debe mostrar por consola:

- El valor exacto del promedio.
- Un mensaje de clasificación basado en la siguiente condición:
  - "Suspenso" si el promedio es menor que 5.
  - "Aprobado" si el promedio es mayor o igual que 5.

Ejercicio 2: Crea un programa en Kotlin que solicite al usuario la entrada de un número entero por teclado. El programa debe validar que el número se encuentre en el rango entre 1 y 99 (ambos inclusive). Posteriormente, debe indicar y mostrar por consola el número de gígitos del número introducido.

BUCLES while y do-while

Son muy similares a los de Java. Kotlin admite los operadores y tradicionales break y continue.

```
fun main(parametro: Array<String>) {
     print(“introduce un numero:")
     val valor = readln().toInt()
     var x = 1
     while (x <= valor) {
          println(x)
          x = x + 1
     }
}
```

```
fun main() {
     print(“introduce un numero:")
     val valor = readln().toInt()

     var x = 1
     do {
          println(x)
          x = x + 1
     } while (x <= valor)
}
```

```
fun main() {
     var i = 1
     while (i <= 5) {
          if (i == 3) {
               break // El bucle se detiene
          }
          println("El número es: $i")
          i++
     }
}
```

```
fun main() {
     var i = 0
     while (i < 5) {
          i++
          if (i == 3) {
               continue // Omite la impresión del 3
          }
          println("El número es: $i")
     }
}
```

BUCLE for

El bucle for recorre todo lo que proporciona un iterador. Esto es equivalente al bucle for-each de Java. Ejemplos:

```
fun main() {
     val nombres = listOf("Ana", "Pedro", "Luis")
     for (nombre in nombres) {
           println(nombre)
     }
}
```

```
fun main() {
     for (i in 1..3) {
          println(i)
     }
     for (i in 6 downTo 0 step 2) {
          println(i)
     }
}
```

Ejercicios propuestos (II)

Ejercicio 3: Crea un programa en Kotlin que solicite al usuario introducir 10 notas de alumnos, una a una. El programa debe procesar las notas y, al finalizar, mostrar por consola:

- La cantidad de alumnos con nota Aprobada (nota mayor o igual a 7).
- La cantidad de alumnos con nota Suspenso (nota menor que 7).

Ejercicio 4: Crea un programa en Kotlin que muestre por consola todos los múltiplos del número 8 hasta el valor 500, ambos inclusive. Los múltiplos deben aparecer separados por un guion (-) en la misma línea, con el siguiente formato: 8 - 16 - 24 - ... - 500.

Ejercicio 5: Crea un programa en Kotlin que solicite al usuario introducir un número entero por teclado. El programa debe validar que el número se encuentre dentro del rango de 0 a 999 y, posteriormente, mostrar un mensaje por consola indicando si el número tiene uno, dos o tres dígitos. El programa debe finalizar cuando reciba un 0 por teclado.

Ejercicio 6: Crea un programa en Kotlin que permita la entrada del peso de varias piezas. El proceso termina cuando introducimos el valor 0. En ese momento se debe mostrar por pantalla:

- Número de piezas con un peso entre 6.5 y 10
- Número de piezas con más de 10
- Número de piezas con menos de 6.5
- La cantidad total de piezas introducidas.

Ejercicios propuestos (III)

Ejercicio 7: Crea un programa en Kotlin que realice un conteo de todos los números enteros comprendidos en el rango de 1 a 100.000 (ambos inclusive) que son simultáneamente múltiplos de 3, de 4 y de 9.

Al finalizar el proceso, el programa debe mostrar por consola el número total de múltiplos encontrados.

Ejercicio 8: Crea un programa en Kotlin que solicite al usuario ingresar N pares de datos (medida de la base y medida de la altura), donde cada par corresponde a un triángulo.

La entrada de datos debe finalizar cuando el usuario lo indique. El programa debe cumplir con los siguientes requisitos:

a) Cálculo y visualización: Por cada par de datos ingresado, debe calcular y mostrar por consola la medida de la base, la altura y la superficie del triángulo. (Recuerda que la superficie se calcula como: Superficie=(Base×Altura)/2).

b) Al finalizar la entrada de datos, debe mostrar la cantidad total de triángulos cuya superficie calculada es mayor a 20.

ARRAYS

Al igual que en Java, los arrays son secuencias de datos, del mismo tipo e identificados por un nombre común. Para acceder aun elemento usaremos el método **get(índice de 0 al tamaño del array)** y para saber el tamaño usaremos el método **size().**

Imaginemos un array de Strings con todos los días de la semana.

```
fun main(args: Array<String>) {
val weekDays = arrayOf("Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo")
     println(weekDays.get(0))
     println(weekDays.get(1))
     println(weekDays.get(2))
     println(weekDays.get(3))
     println(weekDays.get(4))
     println(weekDays.get(5))
     println(weekDays.get(6))
     if(weekDays.size >= 8){
          println(weekDays.get(7))
     }else {
         println("no hay más elementos")
     }
}
```

ARRAYS

Recorrido de arrays. En el ejemplo anterior podemos cambiar la forma de recorrer el array e imprimir sus valores a una forma más eficiente usando bucles.

```
fun main(args: Array<String>) {
     val weekDays = arrayOf("Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo")

     for (day in weekDays) {
         println(day)
     }
}
```

Podemos usar el índice para recorrer:

```
fun main(args: Array<String>) {
     val weekDays = arrayOf("Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo")

     for (posicion in weekDays.indices){
          println(weekDays.get(posicion))
     }
 }
```

ARRAYS

Podemos usar los pares (índice, valor):

```
fun main(args: Array<String>) {
     val weekDays = arrayOf("Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo")

     for ((posicion, valor) in weekDays.withIndex()) {
          println("La posición $posicion contiene el valor $valor")
     }
}
```

Esto sería equivalente a:

```
fun main(args: Array<String>) {
     val weekDays = arrayOf("Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo")

     for (posicion in weekDays.indices){
          println("La posición $posicion contiene el valor ${weekDays.get(posicion)}")
     }
 }
```

Ejercicios propuestos (IV)

<u>Ejercicio 9:</u> Registro de precipitaciones

Crea un Array de tipo Double llamado precipitaciones de tamaño 7 (una semana).

Inicializa el array con siete valores de precipitación (en mm), simulando la lectura manual (puedes usar un bucle for para solicitar los datos o inicializarlos directamente en el código).

Calcula el promedio de precipitación semanal.

Recorre el array nuevamente e imprime, para cada día, la desviación de ese día con respecto al promedio (valor de ese día menos el valor promedio e indica para cada día si está “por encima” o “por debajo” del promedio).

LISTAS

Las colecciones en Kotlin se pueden clasificar en dos grandes grupos, las mutables e inmutables. Es decir, las que se pueden editar (mutables) y las que son solo de lectura (inmutable).

Listas inmutables

Las listas inmutables en Kotlin son colecciones de elementos que **no** se pueden modificar después de su creación. Podemos pues añadir, eliminar y modificar elementos.

Para crear una lista inmutable, se utiliza la función de fábrica **listOf().** Por defecto, esta función devuelve un objeto de tipo List, que garantiza la inmutabilidad.

```
val listaDias: List<String> = listOf("Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo")
```

Listas mutables

Las listas inmutables en Kotlin son colecciones de elementos que si se pueden modificar después de su creación. Podemos pues añadir, eliminar y modificar elementos.

Para crear una lista inmutable, se utiliza la función de fábrica **mutableListOf().** Por defecto, esta función devuelve un objeto de tipo mutableList.

```
val listaDiasMut: mutableList<String> = mutableListOf("Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo")
```

LISTAS inmutables : métodos

- Métodos para obtener elementos:
  - get(index: Int) o la sintaxis de corchetes [index]: Devuelve el elemento en una posición específica.
  - first(): Devuelve el primer elemento de la lista.
  - last(): Devuelve el último elemento de la lista.
- Métodos para buscar elementos:
  - indexOf(element: T): Devuelve la posición de la primera aparición de un elemento, o -1 si no lo encuentra.
  - contains(element: T): Devuelve true si la lista contiene el elemento, de lo contrario, false.
- Métodos para obtener información:
  - size: Devuelve el número de elementos en la lista.
  - isEmpty(): Devuelve true si la lista está vacía.
- Métodos para transformar la lista (devolviendo una nueva lista):
  - subList(fromIndex: Int, toIndex: Int): Devuelve una nueva lista con los elementos del rango especificado.
  - slice(indices: Iterable<Int>): Devuelve una nueva lista que contiene los elementos en las posiciones especificadas.
  - sorted(): Devuelve una nueva lista con los elementos ordenados.

LISTAS inmutables : métodos

```
fun main(args: Array<String>) {
var lista1: List<String> = listOf("lunes", "martes", "miercoles", "jueves", "viernes", "sábado", "domingo")
     println(lista1.isEmpty()) //Salida: false
     println(lista1) // Salida: [lunes, martes, miercoles, jueves, viernes, sábado, domingo]
     println(lista1.get(0)) // Salida: lunes
     println(lista1[0]) // Salida: lunes
     println(lista1.first()) // Salida: lunes
     println(lista1.get(lista1.size-1)) // Salida: domingo
     println(lista1.last()) // Salida: domingo
     println(lista1[lista1.size-1]) // Salida: domingo
     println(lista1.size) // Salida: 7
     for(elemento in lista1)
          print("$elemento ") // Salida: lunes martes miercoles jueves viernes sábado domingo
     println()
     for(posicion in lista1.indices)
           print("[$posicion]${lista1[posicion]} ")
          // Salida: [0]lunes [1]martes [2]miercoles [3]jueves [4]viernes [5]sábado [6]domingo
     lista1.indexOf(“lunes”) //Salida: 0
     lista1.indexOf(“hoy”) //Salida: -1
     lista1.contains(“martes”) //Salida: true
     lista1.contains(“mañana”) //Salida: false
     println(lista1.subList(1, 4)) //Salida: [martes, miercoles, jueves]
     println(diasSemana.slice(listOf(2, 0, 5))) // Salida: [Miércoles, Lunes, Sábado]
     println(lista1.sorted()) // Salida: [domingo, jueves, lunes, martes, miercoles, sábado, viernes]
}
```

Ejercicios propuestos (V)

<u>Ejercicio 10:</u> Búsqueda y Filtrado de Códigos de Productos

Crea una lista inmutable (List<String>) llamada codigosProductos con al menos 10 códigos de producto simulados (ej. "A105-R", "B201-V", "C330-R", etc.).

Utiliza una función de filtrado para crear una nueva lista que solo contenga los códigos que terminan con el sufijo "-R".

Utiliza una función de filtrado para encontrar el primer código que empiece con "B" y tenga más de 6 caracteres. Si no lo encuentra, indica que no se ha encontrado.

Imprime las tres listas: la original, la de reacondicionados y el resultado de la búsqueda.

LISTAS mutables : métodos

- Métodos para añadir y eliminar elementos
  - add(element: T): Añade un elemento al final de la lista.
  - add(index: Int, element: T): Inserta un elemento en una posición específica.
  - addAll(elements: Collection<T>): Añade todos los elementos de otra colección.
  - remove(element: T): Elimina la primera ocurrencia del elemento especificado.
  - removeAt(index: Int): Elimina el elemento en una posición específica.clear(): Elimina todos los elementos de la lista.
- Métodos para actualizar elementos
  - set(index: Int, element: T) o la sintaxis de corchetes [index] = element: Reemplaza el elemento en una posición específica por uno nuevo.
  - sort(): Ordena la lista en su lugar, modificando la lista original.
- Métodos de la interfaz List (lectura).Además de los métodos de modificación, las listas mutables también tienen todos los métodos de lectura de las listas inmutables, como:
  - get(index: Int) o [index]: Devuelve el elemento en una posición específica.
  - size: Devuelve el número de elementos.
  - isEmpty(): Comprueba si la lista está vacía.
  - indexOf(element: T): Devuelve la posición de un elemento.

LISTAS mutables : métodos

```
fun main(args: Array<String>) {
    val listaMutable: MutableList<String> = mutableListOf("Lunes", "Martes", "Miércoles", "Jueves", "Viernes")}
    println("Lista original: $listaMutable") // Salida: [Lunes, Martes, Miércoles, Jueves, Viernes]
    println("Está vacía?: ${listaMutable.isEmpty()}") // Salida: false
    println("Tamaño: ${listaMutable.size}") // Salida: 5
    println("Primer elemento: ${listaMutable.first()}") // Salida: Lunes
    println("Tercer elemento (índice 2): ${listaMutable[2]}") // Salida: Miércoles
    println("Índice de 'Viernes': ${listaMutable.indexOf("Viernes")}") // Salida: 4
    println("Contiene 'Sábado'?: ${listaMutable.contains("Sábado")}") // Salida: false
    println("Lista ordenada: ${listaMutable.sorted()}") // Salida: [Jueves, Lunes, Martes, Miércoles, Viernes]
    listaMutable.add("Sábado") // Añade al final
    listaMutable.add(6, "Domingo") // Añade en una posición específica (índice 6)
    println("Lista después de añadir días: $listaMutable")
    // Salida: [Lunes, Martes, Miércoles, Jueves, Viernes, Sábado, Domingo]
    listaMutable.addAll(listOf("Festivo", "Reserva"))  // Añade la colección al final
    println("Lista después de añadir colección al final: $listaMutable")
    // Salida: [Lunes, Martes, Miércoles, Jueves, Viernes, Sábado, Domingo, Festivo, Reserva]
    listaMutable[4] = "DIA DE TRABAJO“ //Modifica el elemento con índice 4 y le asigna el nuevo valor
    println("Lista después de actializar el element 4: $lista// Reemplaza 'Viernes' (índice 4) Mutable")
    // Salida: [Lunes, Martes, Miércoles, Jueves, DIA DE TRABAJO, Sábado, Domingo, Festivo, Reserva]
    listaMutable.remove("Reserva") // Elimina por valor
    listaMutable.removeAt(7) // Elimina 'Festivo' (índice 7, ya que 'Reserva' se eliminó antes)
    println("Lista después de REMOVE: $listaMutable")
    // Salida: [Lunes, Martes, Miércoles, Jueves, DIA DE TRABAJO, Sábado, Domingo]
    listaMutable.forEachIndexed { posicion, valor ->
        println("[$posicion] $valor")
    }
}
```

Ejercicios propuestos (VI)

Ejercicio 11: Gestión de tareas

Crea una lista mutable (MutableList<String>) llamada listaTareas e inicialízala con 3 tareas.

Solicita al usuario una nueva tarea por teclado y añádela a la lista.

Elimina la primera tarea de la lista.

Modifica la segunda cambiándola por la cadena “Tarea modificada”.

Imprime la lista final de tareas.

LISTAS conversión

Se puede convertir una lista inmutable en mutable con el método ***toMutableList().***

```
val listaDias = listOf("Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo")

// Convertir en mutable
val listaDiasMutable = listaDias.toMutableList()

println(listaDiasMutable)
```